



Golang Gin Backend Pertemuan 18

Membahas logging pada backend Gin, pentingnya pencatatan aplikasi, serta dasar unit testing dengan dua latihan praktik intermediate.



Juara Coding

June 2026

Tujuan Pembelajaran



01 Memahami peran logging

Peserta memahami fungsi logging dalam backend service serta manfaatnya untuk pelacakan dan analisis masalah.

02 Membedakan debugging dan monitoring

Peserta mampu membedakan penggunaan log untuk debugging saat pengembangan dan monitoring saat sistem berjalan.

03 Menulis test dan praktik aman

Peserta dapat menulis unit test sederhana serta menerapkan praktik pengujian agar pengembangan kode lebih aman.



Pengantar Logging Gin

Logging mencatat request, error, dan proses penting agar developer dapat menelusuri alur aplikasi serta memantau sistem.

Manfaat Logging



Logging membantu melacak error, pola akses endpoint, dan respons aplikasi agar debugging serta observability lebih efektif.



Deteksi Error

Mengetahui kapan error terjadi sehingga penanganan bisa dilakukan lebih cepat.



Analisis Endpoint

Melihat endpoint yang paling sering diakses untuk memahami pola penggunaan aplikasi.



Debugging Mudah

Log yang baik mempercepat investigasi insiden dan meningkatkan observability aplikasi.

Logging Bawaan di Gin

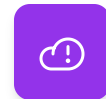


Gin menyediakan middleware `Logger()` untuk mencatat detail request dan biasanya dipakai bersama `Recovery()`.



Mencatat Request

`Logger()` merekam method, path, status code, latency, dan alamat client untuk tiap request.



Dipakai dengan Recovery

`Recovery()` menangani panic agar proses gagal tetap tercatat saat request diproses.



Membantu Monitoring

Kombinasi keduanya memudahkan pelacakan error dan perilaku aplikasi saat berjalan.

```
func main() {  
    r := gin.Default()  
    r.Use(LoggerMiddleware())  
    r.GET("/ping", func(c *gin.Context) {  
        c.String(200, "pong")  
    })  
    r.GET("/hello", func(c *gin.Context) {  
        c.String(200, "Hello, World!")  
    })  
    r.Run(":8080")  
}  
  
func LoggerMiddleware() gin.HandlerFunc {  
    return :  
    return func(c *gin.Context) {  
        start := time.Now()  
        c.Next()  
        latency = time.Since(start)  
        log.Printf("GIN] %s | %d %s | v",
```

Contoh Middleware Logger

gin.Default() otomatis memasang Logger dan Recovery, sehingga request dasar langsung tercatat tanpa konfigurasi tambahan.

Informasi yang Dicatat



Elemen log utama

Catat waktu kejadian, level log, endpoint, status respons, dan request ID untuk analisis yang jelas.



Hindari data sensitif

Jangan mencatat data pengguna yang sensitif agar log tetap aman dan tidak menambah risiko keamanan.



Pesan error dan konteks

Sertakan pesan error serta konteks penting agar proses pelacakan masalah menjadi lebih cepat dan akurat.



Level Logging



Info

Dipakai untuk kejadian normal atau alur aplikasi yang berjalan sebagaimana mestinya.



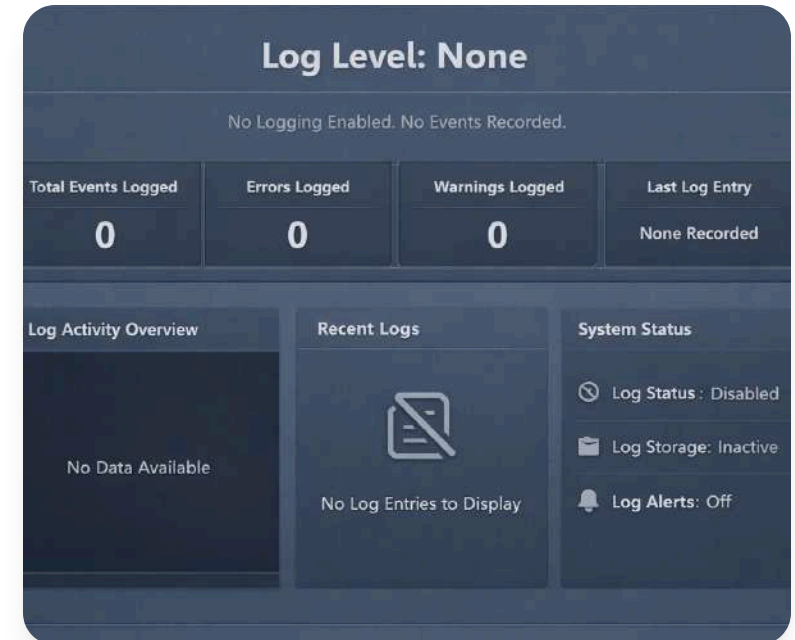
Error dan Debug

Error untuk kegagalan proses, sedangkan debug berisi detail teknis untuk investigasi saat pengembangan.



Warning

Menandai kondisi yang tidak ideal, tetapi sistem masih dapat berjalan.




```
    "log"  
}  
  
func handler(w http.ResponseWriter, r *http.Request) {  
    ctx = r.Context()  
    // Extracting request details  
    method = r.Method  
    path = r.URL.Path  
    userAgent = r.UserAgent()  
    reqID = ctx.Value("requestID")  
  
    // Logging request context  
    log.Printf("Method: %S, Path: %S, User-Agent: %S, RequestID: %s",  
        method, path, userAgent, reqID)  
  
    // Responding to the request
```

Contoh Logging Manual

Logging manual di handler berguna untuk mencatat konteks bisnis tertentu yang tidak direkam otomatis oleh middleware bawaan.

Praktik Logging yang Baik



01 Format Konsisten

Gunakan format log yang seragam agar mudah dibaca, dicari, dan dianalisis di berbagai sistem.

02 Pesan Jelas dan Kontekstual

Tulis pesan singkat namun jelas, serta tambahkan konteks penting agar penyebab masalah lebih cepat dipahami.

03 Catat Hal yang Penting

Pisahkan log informasi dan error, serta hindari logging berlebihan agar analisis tetap fokus dan efisien.

```
}  
  
func TestGetUserByID(t *testing.T) {  
    // Arrange  
    mockDB := &MockUserStore{}  
    mockDB.On("GetUserByID", 1).Return(&User{ID: 1, Name: "Alice"}, nil)  
}  
  
    // Act  
    user, err := mockDB.GetUserByID(1)  
}  
  
    // Assert  
    assert.NoError(t, err)  
    assert.Equal(t, 1, user.ID)  
    assert.Equal(t, "Alice", user.Name)  
}
```

TERMINAL

PROBLEMS

OUTPUT

DEBUG CONSOLE

Pengantar Unit Testing

Unit testing menguji fungsi atau method kecil untuk memastikan hasil sesuai ekspektasi, membantu deteksi bug dini dan refactor aman di backend Go.

Tool Dasar Unit Testing di Go

```
5 func TestAdd(t *testing.T) {
6     result := -Add(2, 3)
7     if result := 5
8         t.Errorf("Expected 5, got %d", result)
9     }
10 }
11
12 func TestSubtract(t *testing.T) {
13     result := -Subtract(7, 4)
14     if result := 3
15         t.Errorf("Expected 3, got %d", result)
16     }
17 }
18
19 $ go test
---FAIL: TestAdd (0.00s)
math_test.go:7: Expected 5, got 4
```

01 Package testing standar

Go menyediakan package *testing* bawaan untuk menulis dan menjalankan unit test secara sederhana.

02 Konvensi file dan fungsi

File test memakai akhiran *_test.go*, dan fungsi pengujian diawali dengan *Test* agar dikenali.

03 Menjalankan pengujian

Test dijalankan dengan perintah *go test* pada package yang ingin diperiksa selama proses pengembangan.

```
}  
  
func TestAdd(t *testing.T) {  
    result := Add(2, 3)  
    if result != 5 {  
        t.Errorf("Expected 5, but got %d", result)  
    }  
}  
  
func TestSubtract(t *testing.T) {  
    result := Subtract(10, 4)  
    if result != 6 {  
        t.Errorf("Expected 6, but got %d", result)  
    }  
}
```

Struktur Dasar Unit Test

Contoh ini menunjukkan cara membandingkan hasil aktual dengan nilai yang diharapkan. Jika berbeda, pengujian akan dinyatakan gagal.

```
}  
  
import "testing"  
  
func TestAdd(t *testing.T) {  
    result := add(2, 3)  
    if result != 5 {  
        t.Errorf("Expected 5, got %d", result)  
    }  
}
```

Contoh Fungsi yang Diuji

Fungsi *Add* yang sederhana cocok untuk mengenalkan konsep unit testing sebelum membahas service, validasi input, atau handler HTTP.

Menjalankan Unit Test

```
--- PASS: TestAddNumbers (0.00s)
    add_test.go:15: Result: 5
=== RUN TestSubtractNumbers (0.00s)
--- PASS: TestSubtract (0.00s)
    subtract_test.go:20: Result: 3
=== RUN TestDivideNumbers (0.00s)
--- PASS: TestDivideNumber (0.00s)
=== RUN TestFailingFunction
--- failing_test.go:12: Expected error, got nil
    failing_test.go:14: Error: division by zero
FAIL
myAPP myapp/calculations 0.003s
ok    myapp/handlers    0.002s
ok    myapp/models      0.001s
=== RUN TestProcessData
--- PASS: TestProcessData (0.00s)
```

01 go test

Menjalankan seluruh test pada package yang sedang aktif.

02 go test -v

Memberikan output lebih rinci agar hasil test lebih mudah ditinjau.

03 go test ./...

Menguji semua package dalam project sekaligus untuk memeriksa seluruh unit test.

Table-Driven Test

Test Case ID	Description	Input	Expected Output	Result
TC-01	Valid Move	Move: E4	Success	Pass
TC-02	Invalid Move	Move: Z9	Error: Invalid move	Fail
TC-03	Capture Stones	Board State: Capture at D5	Captured: 3 stones	Pass
TC-04	Check for Ko	Board State: Ko situation	Ko Detected	Pass
TC-05	Check for Win	Board State: Black wins	Black Wins	Pass

01 Pola umum di Go

Dalam Go, *table-driven test* dipakai untuk menguji banyak skenario dengan format yang rapi dan konsisten.

02 Kasus disimpan dalam tabel

Setiap test case berisi input dan output yang diharapkan, sehingga struktur pengujian lebih jelas dan mudah dibaca.

03 Dijalankan dengan loop

Semua kasus diuji lewat perulangan agar test lebih ringkas, mudah diperluas, dan sederhana untuk dirawat.

```
6     name string
7   }{,
8   {
9     name: "positive numbers",
10    a: 2, b: 3,
11    want: 5,
12  },
13  name: "negative numbers",
14  a: -1, b: -2,
15  want: -3,
16 },
17 name: "zero values",
18 a: 0, b: 0,
19 want: 0,
20 },
21 name: "mixed values",
```

Contoh Table-Driven Test

Pola ini menyusun beberapa skenario uji dalam satu struktur sehingga penambahan kasus baru menjadi lebih mudah dan rapi.

Unit Testing Handler Gin

```
import (
    "net/http/httptest"
    "testing"

    "github.com/gin-gonic/gin"
)

func TestGetUserHandler(t *testing.T) {
    router := gin.Default()
    router.GET("/user/:id", GetUserHandler)

    // Simulate HTTP request
    req := httptest.NewRequest("GET", "/user/42", nil),
    w := httptest.NewRecorder()

    router.ServeHTTP(w, req)

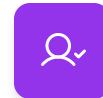
    assert.Equal(t, http.StatusOK, w.Code)
    assert.JSONEq(t, {"id": 42, "name": "John Doe"}, w.Body.String(),
```

Handler Gin dapat diuji dengan *httptest* untuk mensimulasikan request HTTP tanpa server nyata dan memeriksa hasil endpoint secara otomatis.



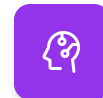
Simulasi Request HTTP

Gunakan *net/http/httptest* untuk menguji handler tanpa menjalankan server sungguhan.



Validasi Hasil Respons

Verifikasi status code dan body respons agar perilaku endpoint tetap sesuai harapan.



Otomatisasi Pengujian

Pengujian membantu developer memastikan endpoint bekerja konsisten secara otomatis.

```
http://127.0.0.1:8080/data
```

```
Response status: 200 OK
```

```
Response body: <nil>
```

```
-----  
PASS: TestLocalEndpoint (0.00s)
```

```
example/testapi 0.001s
```

Contoh Test Handler Gin

Test ini memeriksa respons endpoint */ping* secara lokal dengan membuat request GET dan memvalidasi kode status *200*.

Hal yang Perlu Diuji



Validasi Input

Uji format, tipe data, dan batasan agar input tidak merusak alur aplikasi.



Penanganan Error

Verifikasi error ditangani konsisten agar sistem tetap aman dan mudah ditelusuri.



Logika dan Respons

Pastikan logika bisnis, handler, dan status code menghasilkan keluaran yang tepat.



Hubungan Logging dan Testing

Passed: 289
Failed: 32
In Progress: 4

Test Execution Trend



"Logging membantu saat test gagal, dan unit testing menjaga fungsi tetap konsisten."

Materi Presentasi

Timestamp	Level	Test Case	Message	Details
2023-01-01 14:23 PM	ERROR	LoginTest	Assertion failed: Invalid credentials	View
2023-01-01 14:44:10 PM	ERROR	CheckoutTest	Payment processing error	View
2023-01-01 14:42:07 PM	WARNING	APITest	Response time exceeded threshold	View

Latihan Praktik 1



```
func TestHelloHandler(t *testing.T) {
    router := gin.Default()
    req, _ := http.NewRequest("GET", "/", nil)
    recorder := httptest.NewRecorder()
    router.ServeHTTP(recorder, req)

    assert.Equal(t, http.StatusOK, recorder.Code)
    assert.Contains(t, recorder.Body.String(), "Hello, World!")
}
```

```
INFO(0000) Incoming request
method=GET path=/hello status=200
INFO(0000) Response sent
status=200
PASS: TestHelloHandler (0.001s)
PASS
ok   api/handlers 0.002s
```

```
ERROR(0000) Error fetching user data
ERROR(0000) Database connection failed
DEBUG(0000) Sending response: 500 Internal Server Error
```

01 Buat endpoint GET /products/:id

Gunakan Gin untuk membuat handler yang mengambil parameter *product id* dari URL dan memproses permintaan.

03 Siapkan unit test handler

Buat pengujian untuk memastikan status code **200** pada kasus sukses serta skenario error pada handler tersebut.

02 Tambahkan validasi dan logging

Catat *product id* yang diminta secara manual. Jika id kosong atau tidak valid, kembalikan respons error yang sesuai.

Latihan Praktik 2



01 Fungsi validasi user

Buat validasi sederhana: nama tidak boleh kosong dan umur harus lebih dari 0.

03 Logging validasi gagal

Tambahkan logging untuk mencatat input yang gagal diperiksa selama proses validasi.

02 Table-driven test

Tulis beberapa skenario uji untuk kasus valid dan tidak valid agar perilaku fungsi mudah diverifikasi.



Ringkasan

Logging mencatat aktivitas backend, *testing* memverifikasi fungsi dan handler.